

```

class BST {
    // Node structure
    class Node {
        int key;
        Node left, right;

        Node(int item) {
            key = item;
            left = right = null;
        }
    }
    Node root;
    // INSERT METHOD
    Node insert(Node root, int key) {
        // Step 1: If tree/subtree is empty → create node
        if (root == null) {
            System.out.println("Inserted " + key);
            return new Node(key);
        }
        // Step 2: Compare and go left
        if (key < root.key) {
            System.out.println(key + " < " + root.key + " → go left");
            root.left = insert(root.left, key);
        }
        // Step 3: Compare and go right
        else if (key > root.key) {
            System.out.println(key + " > " + root.key + " → go right");
            root.right = insert(root.right, key);
        }
        // Step 4: Return unchanged root
        return root;
    }
    public static void main(String[] args) {
        BST tree = new BST();
    }
}

```

```
tree.root = tree.insert(tree.root, 50);  
tree.insert(tree.root, 30);  
tree.insert(tree.root, 70);  
tree.insert(tree.root, 20);  
tree.insert(tree.root, 40);  
}  
}
```